Implementation and Comparison of Go-Back-N and Selective Repeat

Raed Ovais

October 16, 2026

1 Approach & Implementation

To evaluate sliding window transport protocols, Go-Back-N (GBN) and Selective Repeat (SR) sender/receiver programs were implemented in Python using UDP sockets (`socket.AF_INET`, `socket.SOCK_DGRAM`). Packet dictionaries containing sequence numbers and payload data were serialized using `pickle` before transmission across `127.0.0.1`.

- **Go-Back-N (GBN):** Uses cumulative acknowledgments. On socket timeout, the sender retransmits the entire unacknowledged window batch from the base packet sequence.
- **Selective Repeat (SR):** Uses individual acknowledgments. The receiver buffers out-of-order packets, and the sender retransmits only the specific missing sequence numbers.

2 Experimental Setup & Results

Both protocols were executed to reliably transfer a workload of 200 packets under identical network loss conditions. Transmission metrics were logged directly from execution output:

Table 1: Protocol Run Statistics (200 Base Packets)

Metric	Selective Repeat (SR)	Go-Back-N (GBN)
Ideal Packets Needed	200	200
Actual Packets Sent	212	696
Retransmission Overhead	12 packets	496 packets
Overhead Percentage	6.0%	248.0%

3 Comparison & Analysis

- **Overhead Difference:** SR required only 12 retransmissions (6.0% overhead) to deliver all 200 packets successfully[cite: 1]. In contrast, GBN triggered 496 total retransmissions (248.0% overhead) due to cascading retransmissions of full windows when a single packet was dropped[cite: 2].
- **Efficiency:** SR significantly reduces total network traffic by selectively resending only unACKed sequence numbers, whereas GBN repeatedly resends already-received packets.

4 Conclusion

Selective Repeat achieves far higher transmission efficiency than Go-Back-N under lossy conditions. While GBN requires simpler receiver state, its window-resending mechanism leads to heavy bandwidth waste compared to SR's targeted retransmissions.